

Assignment 1 - Sorting and Selection

Assignment 1 · Design and Analysis of Algorithms

Damir Satimov · SE-2525 · September 2026

GitHub repository: <https://github.com/SDD3108/DAA1>

1. Scope and asymptotic bounds

The project implements integer MergeSort, randomized three-way QuickSort and iterative QuickSelect. Insertion Sort handles MergeSort leaves of at most 15 elements. All bounds below describe running time as n grows; logarithm bases differ only by a constant factor.

Algorithm	Best case	Average / expected case	Worst case
MergeSort	$\Theta(n \log n)$ Sorted input still merges at every level.	$\Theta(n \log n)$ Random order: linear merge work per level.	$\Theta(n \log n)$ Interleaved halves require nearly full comparisons.
QuickSort	$\Theta(n)$ All keys equal: one partition removes the entire array.	$\Theta(n \log n)$ Distinct keys; expectation over random pivots.	$\Theta(n^2)$ Distinct keys and repeatedly extreme pivots.
QuickSelect	$\Theta(n)$ First pivot's equal range contains k .	$\Theta(n)$ Expected retained size shrinks geometrically.	$\Theta(n^2)$ Extreme pivots repeatedly remove only one item.
Insertion Sort	$\Theta(n)$ Already sorted; one key comparison per insertion.	$\Theta(n^2)$ Uniform random permutation has $\Theta(n^2)$ inversions.	$\Theta(n^2)$ Reverse order shifts every preceding element.

Θ gives matching O upper and Ω lower bounds. The QuickSort best case above allows duplicates; restricted to distinct keys its best time is $\Theta(n \log n)$. For the fixed alphabet $\{0, \dots, 9\}$, three-way QuickSort is $\Theta(n)$: each partition removes a key value and there are at most 10 values. Randomization avoids a systematic sorted-input quadratic case, but does not remove the theoretical worst case.

Space and recursion

MergeSort uses one $O(n)$ helper array and $O(\log n)$ stack space. QuickSort partitions in place and uses $O(\log n)$ stack space by recursing only on the smaller side. QuickSelect uses $O(1)$ auxiliary space and a loop; Insertion Sort uses $O(1)$ space. No recursive MergeSort call allocates an array.

Source and authorship

Requirements source: the supplied "Assignment 1 - Divide and Conquer & Asymptotic Notations" handout by Taubakabyl Nurlybek. This is an AI-generated worked example with measured outputs; the handout restricts AI use to concept explanation and debugging. Optional bonus tasks are outside this implementation.

2. Recurrences and correctness

Master Theorem compares $f(n)$ with $n^{\log_b(a)}$ in $T(n) = aT(n/b) + f(n)$. Rounding and constant-size base cases do not change these asymptotic results.

MergeSort: Master Theorem, case 2

$T(n) = 2T(n/2) + \Theta(n)$ for $n > 15$. Here $a = 2$, $b = 2$, $f(n) = \Theta(n)$, and $n^{\log_2 2} = n$. Thus $f(n)$ matches the critical term: $T(n) = \Theta(n \log n)$. For $n \leq 15$, Insertion Sort costs $O(15^2)$, a constant. The same asymptotic bound applies in every input case because this version does not skip ordered merges.

Balanced QuickSort: Master Theorem, case 2

$T(n) = 2T(n/2) + \Theta(n)$ is the balanced, distinct-key model, ignoring the single removed pivot. Here $a = 2$, $b = 2$ and $f(n) = \Theta(n)$; case 2 yields $\Theta(n \log n)$. Looping over the larger partition changes stack depth, not total work.

A uniform random pivot gives every rank equal probability, including on sorted input. For distinct keys, averaging the recurrence over ranks gives $E[T(n)] = (2/n) \sum_{j=0}^{n-1} E[T(j)] + \Theta(n)$, whose solution is $\Theta(n \log n)$. Repeatedly choosing an extreme rank is still possible and gives $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.

Balanced QuickSelect: Master Theorem, case 3

$T(n) = T(n/2) + \Theta(n)$. Here $a = 1$, $b = 2$ and $f(n) = \Theta(n)$, while $n^{\log_2 1} = 1$. The linear term is polynomially larger ($\epsilon = 1$), and $af(n/b) = f(n)/2$ satisfies regularity with $c = 1/2$ for a linear model. Case 3 gives $T(n) = \Theta(n)$, unlike case 2 for the sorts.

The random-pivot algorithm does not guarantee a half split. A pivot in the middle half occurs with probability about $1/2$ and leaves at most $3n/4$ elements; expected work between such reductions is $O(n)$, so summing geometric sizes gives expected $O(n)$. The first partition costs $\Omega(n)$, making the expected bound $\Theta(n)$.

Insertion Sort: decreasing-size recurrence

Worst case: $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Best case: $T(n) = T(n-1) + \Theta(1) = \Theta(n)$. These are subtraction recurrences, not $aT(n/b)$ with constant $b > 1$; the Master Theorem does not apply.

Why the algorithms are correct and stack-safe

MergeSort recursively sorts both halves; each merge writes the smaller available head, preserving sorted order and every input element. Partition maintains four regions: values below the pivot, equal values, unclassified values, and greater values. When a greater value is swapped from the right, the current index stays in place until its replacement is classified.

QuickSort sorts the two strict regions and skips the equal region. The smaller recursive side has at most $(m-1)/2$ elements, so each nested frame halves the problem size and depth is $O(\log n)$, even with quadratic total work. QuickSelect retains only the region containing absolute index k ; if k lies in the equal region, the pivot value is the answer.

3. Measurement and performance

For each algorithm: $n = 1,000; 10,000; 100,000; 1,000,000$, with full-range random integers, sorted random integers, and duplicates in $0 \dots 9$. Each of the 36 cases has two unrecorded warm-ups and five recorded runs on fresh copies of one seeded input. QuickSelect selects $k = n/2$.

The summary selects the median-time run and its own comparison and depth counts. Raw data preserves all 180 runs and seeds. A comparison is one `Integer.compare` between keys. Timing includes algorithm instrumentation and the MergeSort buffer allocation, but excludes input creation/copying, reference sorting and correctness checks. Every run is verified.

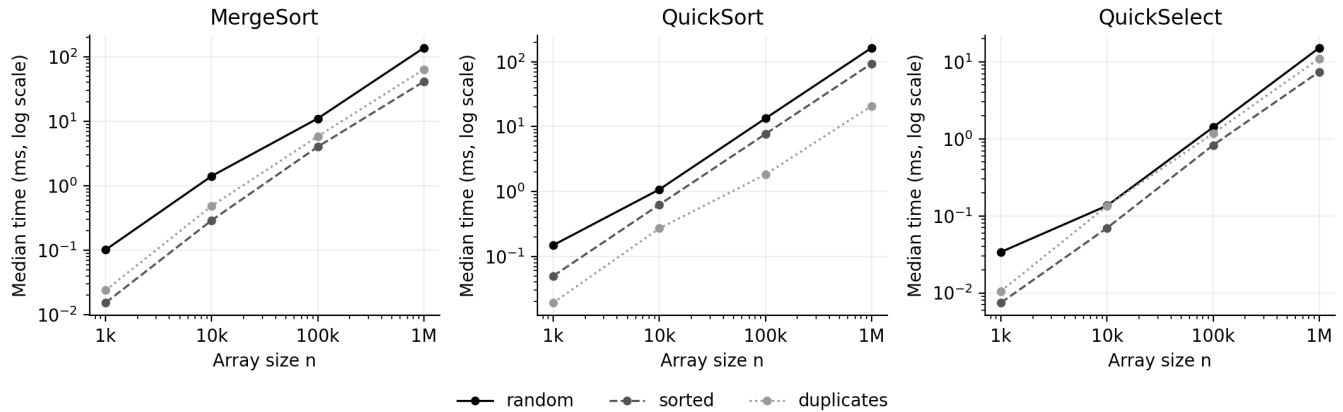


Figure 1. Median elapsed time, with logarithmic x and y axes. Each algorithm/input pair has its own line. Small cases are strongly affected by runtime overhead; these measurements describe this instrumented implementation, not uninstrumented library performance.

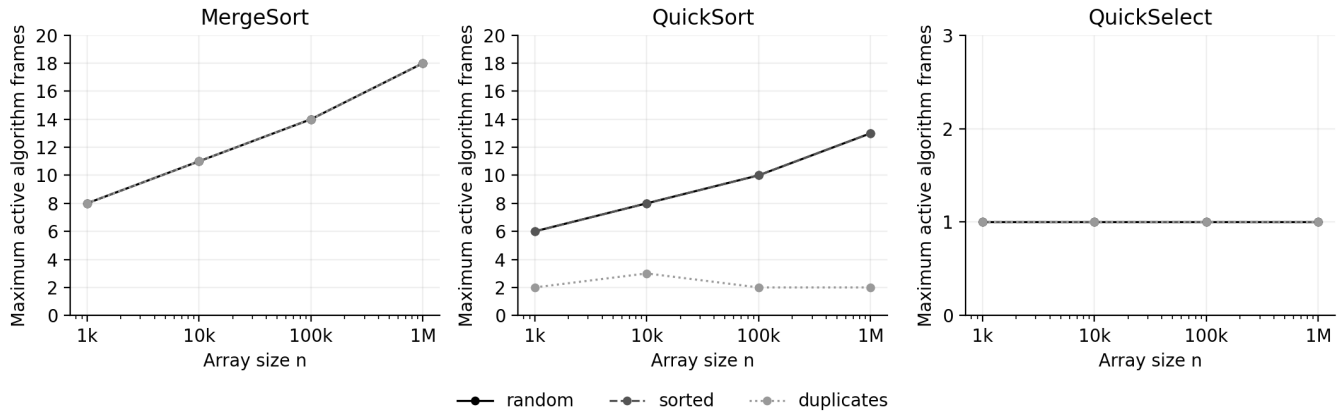


Figure 2. Maximum simultaneous algorithm frames; helper calls do not count. Empty sorts have depth 0 and a nonempty root has depth 1. Iterative QuickSelect stays at 1; it does not accumulate recursive frames as it narrows the active range.

Measured maximum depths across the summary are 18 for MergeSort, 13 for QuickSort and 1 for QuickSelect. The independent sorted-array check uses $n = 100,000$ with ten seeds and requires $\text{maxDepth} \leq 2 \log_2(n) \approx 33.22$. All checks also ran with a 256 KiB Java thread stack.

Environment. Runtime: 17.0.20+8-1-24.04-Ubuntu; VM: OpenJDK 64-Bit Server VM; OS: Linux amd64; Available processors: 8; Maximum heap bytes: 1073741824. Two warm-ups mitigate but do not eliminate JIT compilation, scheduling and garbage-collection effects.

4. Empirical Θ check

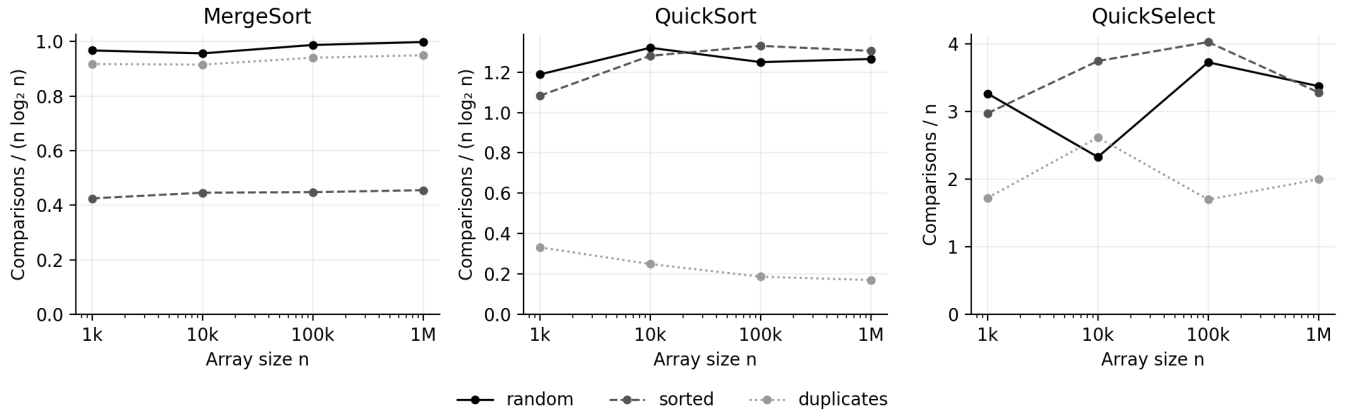


Figure 3. $C(n)/(n \log_2 n)$ for the sorts; $C(n)/n$ for selection, where C is measured key comparisons. MergeSort and distinct-key QuickSort show nearly stable ratios; QuickSelect stays on a constant scale but has substantial pivot-dependent fluctuations.

The table gives observed ratio ranges over $n \in \{10,000, 100,000, 1,000,000\}$. Candidate constants use a 15% margin, rounded outward to 0.01. With $n_0 = 10,000$, $c_1 g(n) \leq C(n) \leq c_2 g(n)$ holds for those **measured points**. Finite samples cannot establish the formal claim “for all $n \geq n_0$ ”; the recurrences and probabilistic analysis supply the asymptotic argument.

Algorithm / input	$g(n)$	Observed ratio	c_1	c_2
MergeSort / random	$n \log_2 n$	0.956 - 0.998	0.81	1.15
MergeSort / sorted	$n \log_2 n$	0.446 - 0.455	0.37	0.53
MergeSort / duplicates	$n \log_2 n$	0.915 - 0.949	0.77	1.10
QuickSort / random	$n \log_2 n$	1.249 - 1.320	1.06	1.52
QuickSort / sorted	$n \log_2 n$	1.280 - 1.329	1.08	1.53
QuickSelect / random	n	2.329 - 3.731	1.97	4.30
QuickSelect / sorted	n	3.285 - 4.031	2.79	4.64
QuickSelect / duplicates	n	1.701 - 2.618	1.44	3.02

Why the duplicate-heavy QuickSort line falls

Its plotted ratios are 0.332, 0.250, 0.187, 0.171. They decline rather than approach a positive $n \log n$ constant. With only ten possible values, each element participates in at most ten partitions: $n \leq C(n) \leq 10n$, so the appropriate tight bound for this input family is $\Theta(n)$.

The alternative normalization $C(n)/n$ ranges from 3.105 to 3.399 over these four sizes. The theoretical constants $c_1 = 1$, $c_2 = 10$ and $n_0 = 2$ are valid for this implementation on all such arrays. For randomized distinct-key QuickSort and QuickSelect, empirical bands are consistent with expected bounds; they are not guarantees for every pivot sequence.

5. Results, validation and discussion

Measured results at $n = 1,000,000$

Algorithm	Random: ms / comparisons	Sorted: ms / comparisons	Duplicates: ms / comparisons
MergeSort	136.990 / 19,886,588	41.216 / 9,071,040	63.563 / 18,919,820
QuickSort	163.456 / 25,202,544	93.574 / 26,003,594	20.834 / 3,399,309
QuickSelect	15.104 / 3,377,610	7.404 / 3,284,873	10.977 / 1,999,512

Verification evidence

Both sorting algorithms were checked against `Arrays.sort` on 150 random arrays each, plus empty, singleton, equal, sorted, reversed, extreme-integer and cutoff-boundary inputs. QuickSelect was checked at every rank on 120 random arrays, plus edge cases and invalid arguments. Ten sorted arrays of 100,000 elements met the QuickSort depth bound. Partition invariants, permutation preservation, counter reset and known comparison counts were also checked.

An adversarial deterministic pivot sequence on 2,000 distinct descending values required $2,000 \times 2,001 / 2 - 1 = 2,000,999$ comparisons while retaining depth 1. This confirms that bounded recursion does not imply bounded runtime. All 12 shared check groups passed, and all 252 benchmark executions (72 warm-ups plus 180 measurements) passed reference verification.

Execution limitation: the Java core and dependency-free verification runner were compiled and executed. JUnit 5 test adapters are supplied, but Maven/JUnit were unavailable in this environment, so no JUnit-engine pass is claimed. Run **mvn clean test** locally before submission.

Discussion (9 sentences)

The comparison ratios support the predicted $n \log n$ growth of MergeSort and expected $n \log n$ growth of distinct-key QuickSort. QuickSelect's comparisons remain proportional to n in scale, although individual pivot sequences make its ratio fluctuate. Duplicate-heavy QuickSort follows linear growth because equal values are removed together. MergeSort performs fewer comparisons on sorted input but still copies and merges at every level. JVM warm-up can change timings through class loading and JIT compilation, so two discarded runs and five measured runs reduce rather than eliminate this effect. Garbage collection and shared-machine scheduling can produce outliers, which makes a median more robust than one timing. CPU caches and branch prediction can change constants even when comparison counts follow the same asymptotic law. The cutoff of 15 saves recursive-call overhead by using Insertion Sort on small ranges. Counting comparisons adds instrumentation overhead, so absolute times should not be interpreted as universal algorithm rankings.

Reproducibility and release

Use **mvn compile exec:java** for the benchmark, or the supplied Python launcher with a JDK when Maven is unavailable. The CSV files, plots, generator, actual check logs and environment details are included. Local Git history contains the four required feature branches and tag v1.0; it is preserved in `history.bundle`. GitHub publishing and Moodle submission remain user actions; the real repository URL must be added to `SUBMISSION.md`.